

Chef's Mind AI — MASTER_STRATEGY v2.2 (Project Plan)

Overview and Current Architecture

Project Mission: Chef's Mind AI is a multi-agent platform to assist restaurant operations (menu planning, purchasing, analytics, content creation) via a unified Orchestrator API and specialized agents ¹. The system architecture is layered as follows: an **Orchestrator** module routes user requests to the appropriate **Agent** (Chef, Accountant, Researcher, Media Studio) and processes responses through a **QA-Gate** middleware for quality control before returning results ². Under the hood, the backend is an Express/TypeScript API, the frontend is Next.js 14 (React 18 + Tailwind), and data is stored in PostgreSQL using Drizzle ORM for migrations. Prometheus metrics and logging are integrated for observability ³ ⁴.

Current Implementation Status: As of version 2.1.4, the core architecture is ~90% complete and stable. All major components are in place and have been validated with smoke tests. The multi-agent routing (Block 2) has been fully implemented and tested: the Orchestrator can intelligently classify user intents into one of 5 categories and route to the corresponding agent ⁵. The enhanced chat endpoint (`POST /api/enhanced-agent/chat`) is active and uses the new orchestrator, achieving ~95% routing accuracy with low latency ⁶ ⁷. A caching layer (in-memory LRU) improves performance for repeated queries. The QA-Gate is integrated into the response flow to perform basic answer validation (checks for placeholders, minimum length, etc.), ensuring a quality score is attached to each answer ⁸ ⁹. On the frontend, the application shell is up with a multi-agent chat UI; a visual indicator shows the active agent responding (Agent Badge) ¹⁰.

Agents and Responsibilities: The platform supports four primary user-facing agents plus an internal QA layer ¹:

- **Chef** – Focuses on culinary tasks: generating recipes, menu plans, and branded food content (including images of dishes).
- **Accountant** – Handles cost analysis, inventory and procurement, interfacing with Google Sheets/Docs/Calendar for data management.
- **Researcher** – Performs research and analytics: fetching relevant information or market data and summarizing insights.
- **Media Studio** – Specializes in media generation (images or short videos) using AI (DALL-E 3, GPT-4 Vision, etc.), with fallback logic if a provider is unavailable.
- **QA-Gate** (middleware) – Not a user agent but an automated quality checker that fact-checks and validates agent outputs, possibly using an LLM tool for verification.

Each agent is implemented as part of the server's **graph workflow**, and the Orchestrator node decides which agent (or sequence of agents) should handle a given request. The agents have been successfully integrated, and the Orchestrator's intent-classification uses simple heuristic or rule-based matching as of now (e.g. keyword matching for finance vs. cooking queries). **Future Enhancement:** A more advanced LLM-based planner for routing is under consideration to improve context-sensitive agent selection ¹¹, but the current heuristic approach is functional and fast.

Authorization and Security

User Roles & RBAC: Role-Based Access Control is implemented to secure sensitive endpoints. There are three defined roles for users: "admin", "chef", and "accountant" ¹². By policy, an admin has access to all endpoints (admin is a superset role), while the chef role would only permit chef-specific operations, and accountant role only accountant-specific operations ¹³ ¹⁴. In practice, most write or admin endpoints are restricted to admin role for now, since the Chef/Accountant roles correspond more to agent operations than to human user accounts. The RBAC middleware (requireAuth and requireRole) is applied globally to any data-modifying API routes. If a user lacks the required role, the request is rejected with 403 Forbidden and an audit log is appended (all access attempts, granted or denied, are logged to a JSON file logs/rbac_smoke.json ¹⁵ ¹⁶).

Authentication & OAuth: User authentication is accomplished via Google OAuth2 integration. A user initiates OAuth login via the /auth/google/start endpoint, and upon Google callback the server stores OAuth tokens in a session ¹⁷ ¹⁸. Logged-in users get an HTTP session cookie (with secure settings in production). The system uses these Google credentials for accessing Google APIs (Docs, Sheets, Calendar) on behalf of the Accountant agent. The OAuth flow on frontend is supported by a **RBACGuard** component that conditionally renders UI elements based on user auth status and role ¹⁹ ²⁰. **Current status:** OAuth integration is partially complete – the frontend and environment are configured for Google OAuth (client IDs, secrets, scopes are set in .env and .env.example), and backend OAuth routes (login, callback, logout, status) have been implemented ²¹ ²². The system can successfully obtain and store OAuth tokens and provide a basic status (including which scopes are granted) at GET /auth/google/status ²³. Remaining work is to rigorously test the OAuth flow end-to-end and handle any edge cases (token refresh, error handling).

Safe Mode (Write Protection): In development and even in production by default, **Safe Mode** is enabled as an extra safety switch for any write operations. When SAFE_MODE=on, any route that modifies data (database writes, file uploads, etc.) requires a special confirmation code in the request (header X-Confirm-Code or in the body) that must match the server's CONFIRM_CODE ²⁴ ²⁵. Without this code, the server will reject the request with a 403 explaining that confirmation is needed. This mechanism prevents accidental or unauthorized destructive actions and is especially useful during development and demo sessions. All key modifying endpoints (import, database DDL, backup/restore, calendar event creation) are wrapped with the requireWriteConfirm middleware in addition to RBAC ²⁶ ²⁷. In practice, the development workflow with Kilo Code (our AI coding assistant) also respects this – it will prompt for confirmation or simulate the header when executing such actions. The Safe Mode status can be checked via a health route /api/safe (returns whether safe mode is on).

Security Hardening: Standard security middlewares are in place: Helmet for basic HTTP headers, CORS restrictions (by default only allowing http://localhost:3000 and 3001 origins for frontend, and configured via env CORS_ORIGIN), and express-session for user sessions with secure cookies. Rate limiting is configured (see server/config/rateLimit.ts) to prevent abuse of certain endpoints (e.g., login attempts, media generation) – currently defaulting to a conservative limit (such as 100 requests/minute per IP, adjustable). These measures ensure the application is robust against common web vulnerabilities. Future steps include enabling HTTPS in production, thorough OAuth security review (CSRF protection on the OAuth flow), and possibly implementing a more fine-grained permission system if new user roles are introduced.

Base Systems (Data & Core Infrastructure)

Database & Migrations: Chef's Mind AI uses **PostgreSQL 15** as the primary datastore. All core entities (ingredients, recipes, suppliers, units, etc.) are defined via Drizzle ORM schema in TypeScript (`shared/schema.ts`) and managed through migration scripts. A set of baseline tables for recipes, ingredients, etc., was established early on, and since then we have been consolidating schema changes into Drizzle migrations rather than runtime DDL. *Progress:* Most of the schema is migrated to Drizzle (units, suppliers, categories, ingredients, prices, recipes, recipe_components, cost_snapshots are in migrations) ²⁸ ²⁹. A previous risk of having “two sources of truth” (some tables created via code at startup and some via migrations) is being resolved – the plan is to deprecate any runtime DDL in `dbadmin.ts` and rely solely on migrations ³⁰. Upcoming schema additions for **Block 3** (see Roadmap) include tables for chat history, orders, etc., which will be added via new migrations (ensuring schema consistency). The app provides a `npm run migrate` command to apply migrations easily ³¹.

Data Access: The backend follows a read/write segregation principle for database access: e.g., using distinct pools or functions `dbRead` vs `dbWrite` (this ensures that certain operations can be easily isolated or even directed to read replicas if needed in the future) ³². The code uses parameterized queries or an ORM for safety – user inputs in queries are validated and sanitized (often through Zod schemas before hitting the DB). For example, the import route uses strict Zod schemas to validate CSV data before constructing SQL inserts ³³.

Infrastructure & Environment: For development, the project provides a Docker setup for PostgreSQL. Developers can spin up the DB with one command and run the API locally. Node.js 20.x and npm 10.x are required on the host machine ³⁴. Key environment variables are defined in `.env.example` for quick setup ³⁵ ³⁶. These include database URLs, API keys for OpenAI and Google, OAuth client IDs, and configuration flags (e.g., `SAFE_MODE`, `MEDIA_PROVIDER_DEFAULT`). Environment loading is validated at runtime: a Zod schema in `server/config/env.schema.ts` ensures critical env vars are present and correctly formatted, preventing the app from starting with misconfiguration. The **startup order** of services is strictly defined to avoid race conditions: first the database, then the backend API, then the frontend ³⁷ ³⁸. This is documented and also enforced in practice by health checks (the frontend won't fully initialize until the API's `/health` is OK, etc.).

Dev Ports and Directory Structure: By team convention, we have standardized development ports to avoid conflicts. The API runs on port **5003** by default (after a recent update from 5001 to avoid a clash) ³⁹ ⁴⁰. The primary frontend uses port **3000** (with an alternate on 3001 for a secondary UI or enhanced version), PostgreSQL uses 5432, and Prometheus metrics are exposed on 9090 ⁴¹. These port settings are reflected in config files and `.env` samples. The project's directory layout is organized for clarity:

```
Chefs-Mind-AI/
├── server/                # Express backend source
│   ├── index.ts          # Server entry point (sets up express app)
│   ├── routes.ts         # Main router aggregator (mounts all route modules)
│   ├── routes/           # API route handlers (grouped by feature)
│   ├── middleware/       # Express middleware (auth, RBAC, SafeMode, QA,
etc.)
│   ├── graph/            # Agent orchestration logic (graph nodes,
orchestrator)
│   └── services/         # External integrations (OpenAI API, Google API,
```

```

etc.)
|   ├── config/          # Configuration (env schema, rate limits)
|   └── utils/           # Utility functions (e.g., logging, helpers)
├── frontend-enhanced/   # Next.js frontend app (dashboard UI for agents)
├── shared/              # Shared TS types and DB schema definitions
├── drizzle/             # Database migration files and config
├── tests/               # Automated tests (unit, integration, E2E)
└── reports/             # Documentation artifacts, logs, action logs, etc.

```

This structure ensures a clear separation of concerns: routes vs. middleware vs. agent logic vs. services, etc. It also helps new contributors quickly locate relevant files.

Local Deployment and Development: To run the project locally for development, follow these steps (after cloning the repo and installing dependencies):

1. **Start the Database** – e.g. using Docker:

```
docker-compose -f docker-compose.prod.yml up -d db
```

Ensure PostgreSQL (5432) is up and running. Optionally, check the container status with `docker-compose ps`.

2. **Apply Migrations** – run the migration script to create/update DB schema:

```
npm run migrate
```

This will run all Drizzle migrations against the local DB. You should see logs of applied migrations.

3. **Launch the Backend API** – in the project root:

```
npm run dev:server
```

This starts the Express server in watch mode. By default it will listen on **http://localhost:5003** (make sure your `.env` has `PORT=5003`). You should see a startup log including a line like `Listening on port 5003`. Test it by visiting **http://localhost:5003/health** which should return `{"ok":true,...}` ⁴² ⁴³.

4. **Launch the Frontend** – in a separate terminal, navigate to the frontend directory and start the Next.js dev server:

```
cd frontend-enhanced
npm run dev
```

This will start the frontend on **http://localhost:3000** (and also on 3001 as a backup). It will proxy API requests to port 5003 automatically (ensure `NEXT_PUBLIC_API_BASE` or similar is set to `http://localhost:5003`). Now you can open the app in your browser at localhost:3000. Logging in via Google will require you to have configured the OAuth client properly in the env variables and Google console.

5. **Troubleshooting:** If the API fails to start, double-check that the database URL in `.env` is correct and the DB is running. If the frontend shows a blank page or cannot reach the API, verify that `NEXT_PUBLIC_API_BASE` is aligned with the backend port (and that CORS allows the frontend origin). Basic connectivity can be verified with `curl http://localhost:5003/health` and from the frontend dev server console logs.

These steps (DB → API → Frontend) reflect the critical startup order ⁴⁴ ⁴⁵. The included **RUNBOOK.md** contains more detailed setup and troubleshooting info if needed.

Analytics and Observability

Application Metrics: Monitoring is built in via Prometheus metrics and logging. The API exposes an endpoint `/metrics` (by default at port 5003) that provides Prometheus-formatted metrics on HTTP requests, performance, etc. This includes custom metrics like request latency (p95, p99) and domain-specific metrics such as AI response times or backup sizes ³ ⁴. We have implemented histograms/summaries for key operations (e.g., `dbQueryLatencySummary`, media generation latency, etc.) and confirmed they work by running smoke tests that produce these metrics ³. Alerts can be configured in Prometheus – an example alerting config has been drafted (e.g., to alert if p95 latency exceeds a threshold) ⁴⁶. A future goal is to set up a Grafana dashboard to visualize these metrics and ensure we have alerts on critical events (like an agent failing to respond, or external API quotas being hit).

Logging and Auditing: The project enforces an **append-only logging policy** for important logs and documentation, meaning we do not overwrite historical logs – we only add new entries. All code or documentation changes executed via the AI assistant (Kilo Code) are recorded in `reports/ACTION_LOG.md` with a timestamp, author, and summary of the change ⁴⁷ ⁴⁸. Additionally, any configuration changes get an “Addendum” section appended to relevant docs rather than editing the original text (this ensures traceability of changes). For example, when development ports were standardized and confirmed, an addendum was added to this effect, and the change was logged ⁴⁹ ⁵⁰. The system also saves artifacts of significant changes under `reports/artifacts/YYYY-MM-DD/HHMM/` (for instance, diffs, copies of updated files, test logs). This provides a paper trail for debugging and code reviews.

At runtime, the server logs to console (which in dev goes to the terminal, and in production can be collected by a process manager). Important security-related events are logged to files as mentioned (RBAC denials, Safe Mode violations). There is also a dedicated `qa.log` for QA-Gate results and a `logs/` directory for other debug logs (like `rbac_smoke.json`).

Known Issues & Performance: The team has identified some environment-specific issues during development, particularly on Windows/WSL. File system operations can behave unexpectedly (e.g., file locks, path issues) when using Docker volumes or WSL interoperability ⁵¹. We have documented these in the Runbook under “Known Issues – File Writes” ⁵¹, and provided workarounds (such as running migrations sequentially to avoid DB locks ⁵², using `npm run clean` to clear caches if build artifacts cause conflicts, etc.). These constraints are largely relevant to development mode and are mitigated by following the recommended steps. In production (Linux container), we expect fewer of these issues, but

we remain cautious by keeping Safe Mode on and thoroughly testing backup/restore functionality on real files.

Performance-wise, the system is currently suitable for pilot use: median API response times are low (the heavy lifting is done by external calls to OpenAI/Google APIs which are the main contributors to latency). The orchestrator classification takes ~50ms average ⁵³, and streaming of responses is supported (though we currently default to a non-streaming mode for stability). We will continue to monitor and optimize as we integrate more features (e.g., the chat history and new CRUD operations might necessitate query optimizations or indexing, which we will address with proper DB schema design).

Routing & API Endpoints

All API endpoints are prefixed under logical groupings. Below is a summary of key routes in the system and their status:

- **Health Check:** GET /health – returns { ok: true, uptime: ... }. This is publicly accessible (no auth) and is used by deployment scripts and the frontend to check if the backend is alive ⁵⁴.
- **Enhanced Agent Chat:** POST /api/enhanced-agent/chat – the unified chat endpoint. The client sends a message and optional parameters, and the orchestrator handles forwarding to the appropriate agent(s). Returns a JSON response containing the agent's reply and metadata (including which agent was used, any QA-Gate score, etc.). **Status:** Implemented and fully working with streaming support via SSE (server-sent events) as needed.
- **Media Generation:** POST /api/media/image/generate and POST /api/media/video/generate – endpoints for the Media Studio agent. They accept a prompt (and for video possibly some options) and respond with either a generated media URL or a status (for asynchronous video jobs). If certain media providers are disabled (e.g., video generation), the endpoint gracefully returns a response indicating that (e.g., { ok: false, reason: "video_disabled" }). **Status:** Image generation is implemented (using the default provider, e.g., OpenAI's DALL·E, with fallback to a secondary model if configured). Video generation is stubbed to return a disabled status unless the ENABLE_SORA flag is true (Sora v2 is an experimental video generator).
- **Import Data:** POST /api/import/upload – allows batch import of CSV or HTML table data for various entities (ingredients, prices, recipes, etc.). It requires admin role + Safe Mode confirm ⁵⁵ ⁵⁶. The endpoint parses the uploaded file and performs upsert operations into the corresponding tables. Strict validation and whitelisting are in place (only specific tables and columns are allowed) ⁵⁷ ⁵⁸. **Status:** Implemented and tested for CSV inputs – returns a summary of inserted/updated records or errors for invalid rows.
- **Database Admin:** Under /api/db/* – includes:
 - POST /api/db/backup – triggers a manual backup of the PostgreSQL database, producing a .sql.gz dump saved to out/backups/ on the server. Also calculates a SHA256 checksum for integrity ⁵⁹ ⁶⁰.
 - GET /api/db/backups – lists available backup files and their checksums.
 - POST /api/db/restore – restores the database from a given backup file (requires providing the filename and matching SHA256 for safety).All these are admin-only and Safe Mode protected ²⁶. **Status:** Implemented and tested (a smoke test covers creating a backup and then restoring from it in a controlled environment ⁶¹).
- **Calendar Events:** POST /api/calendar/payment, /delivery, /followup – creates Google Calendar events for specific purposes (payment due reminder, delivery schedule, follow-

up reminder respectively). Each expects a JSON body with at least a `startTime` (ISO date string) and an optional description ⁶² ⁶³. These endpoints are admin-protected (since they perform actions on an integrated Google account) and require Safe Mode confirmation ⁶⁴. Under the hood, they call the Google Calendar API via our `createEvent` service to actually insert an event in the user's primary calendar with predefined reminders (-24h and -1h) ⁶⁵ ⁶⁶. **Status:** Implemented and tested (when OAuth is set up and valid). There is also a test endpoint `POST /auth/google/test-event` for developers to create a generic test event after logging in, to verify OAuth integration ⁶⁷ ⁶⁸.

- **Google OAuth Routes:** Under `/auth/google` – includes `GET /auth/google/start` (redirects to Google's consent page) ⁶⁹, `GET /auth/google/callback` (handles Google's redirect and stores tokens) ¹⁷, `GET /auth/google/status` (returns JSON with current auth status and scopes) ²³, and `POST /auth/google/logout` (revokes the session) ⁷⁰. **Status:** Implemented; still needs thorough testing and perhaps UI flow integration (front-end likely opens a new tab to start the OAuth flow and then polls the status).

All routes are mounted in the central router (`server/routes.ts`), and we have confirmed that none are missing – earlier issues with routes not being registered have been resolved ⁷¹ ⁷². Notably, the metrics route is mounted only once to avoid duplicate entries ⁷³. Each protected route has appropriate middleware in the chain: JWT authentication (for session/token verification), then `requireAuth` to ensure a user object is attached, `requireRole([...])` if role-specific, and `requireWriteConfirm` if it's a modifying action ²⁶. This layered protection has been validated by comprehensive smoke tests (including an RBAC test suite that tries accessing each write endpoint with and without proper credentials) ⁷⁴.

Roadmap: Next Steps to v2.2

With the core framework in place, the focus of **Master Strategy v2.2** is to finalize remaining features and polish the system for a production-ready state. The development roadmap is organized into execution blocks. Below are the upcoming blocks and tasks:

Block 3: Data Persistence & Calendar (In Progress)

Goal: Implement persistent storage for user-generated data and integrate remaining calendar features. This elevates the system from a stateless demo to a stateful app that can retain information across sessions. Target completion date is **2025-11-22** (ETA) ⁷⁵. Key sub-tasks:

- **Chat History Storage:** Develop a mechanism to save conversation history. Introduce a new `chat_sessions` table (and related tables if needed, e.g., messages) to store chats per user. Create a new endpoint `GET /api/chat/history` to retrieve the last N sessions for the authenticated user (linking sessions to the user's Google OAuth ID or email). This allows the UI to display past conversations and resume context if needed.
- **Orders & Suppliers CRUD:** Implement basic create/read/update/delete endpoints for managing Orders and Suppliers data. This likely entails adding new tables (`orders`, possibly `order_items`, etc., and extending `suppliers` if necessary) via Drizzle migrations. Provide API routes under `/api/orders` and `/api/suppliers` with appropriate RBAC (e.g., only `accountant` or `admin` roles can modify). Additionally, add UI components to list and edit this data (e.g., a simple form or page in the frontend for orders). This feature will connect the dots for inventory management and cost tracking.
- **Calendar Invite Enhancement:** Extend the calendar integration beyond the basic three event types. Possibly create a general endpoint `POST /api/calendar/create` that can handle

various event types with a payload (combining the logic of payment/delivery/followup into one flexible function). Ensure that for any calendar event creation, the OAuth token is valid – if not, gracefully prompt re-authentication.

- **Backup/Restore UI:** Since backup and restore functionality exist at the API level, expose them in the frontend for admin users. For example, add an “Admin” section where an authorized user can click “Create Backup” (which calls the `/api/db/backup` endpoint and then maybe presents a link to download or a success message) and “Restore from Backup” (perhaps allow uploading a backup file or selecting from available backups returned by `/api/db/backups`, then calling restore). This will facilitate easier data resets during testing and is a good admin tool.
- **Documentation & Checkpoint:** Update documentation to reflect these new features. A design doc `docs/DATA_PERSISTENCE_DESIGN.md` will be written to describe how chat history and CRUD are implemented (schema, endpoints). Also, once Block 3 is completed, prepare a new checkpoint file (e.g., `CHECKPOINT_v2.1.5.json`) capturing the updated state (this includes updating any memory bank or context files, and summarizing progress).

Status: Block 3 is currently underway. The groundwork (OAuth, calendar endpoints) is largely done; database migrations for new tables and the corresponding code are the main focus. No blockers anticipated beyond routine schema design and testing. Once complete, the project is expected to reach ~92% readiness ⁷⁵, and we will proceed to the final block.

Block 4: Finalization & Production Prep

Goal: Polish the project, resolve remaining risks (P1/P2 items), and get to a production-ready version 2.2. This phase will likely bring the project from ~92% to 100% feature-complete. Tentative tasks include:

- **Complete DB Consolidation:** Finish migrating any residual runtime DDL to formal migrations (resolving Risk #3) ³⁰. This includes removing or disabling the dynamic DDL execution in `dbadmin.ts` for creating base tables (if it's still present) and ensuring all tables exist via migration files. Perform a fresh database initialization test to confirm we can go from zero to full schema using only migrations and seed data.
- **Advanced Agent Orchestration (Optional):** If time permits, revisit the agent routing logic to incorporate context-aware planning (Risk #8) ⁷⁶. This would involve using an LLM call to decide agent sequence for complex queries (e.g., a request that might need Chef then Accountant). While not strictly required for v2.2 (the heuristic works well now), this could improve system intelligence and is a distinguishing feature. This task would be carefully sandboxed and heavily tested due to its experimental nature. We may defer it to a later version if it risks timeline.
- **Integration Testing & QA:** Write additional tests for all new features from Block 3. For instance, ensure that creating an order and retrieving it works, that chat history is correctly saved and isolated per user, and that the calendar endpoints effectively create events in a live Google Calendar when valid credentials are present. Conduct an end-to-end manual test simulating a user workflow: login, ask Chef agent something, get a response, upload a CSV via the import, see data update, create an order, schedule a delivery (calendar), backup the data, etc., to confirm all parts work together. Address any bugs uncovered.
- **Performance Tuning:** Do a pass on performance and scalability. For example, add DB indexes for any new tables (like chat_sessions on user_id, or orders on date, etc.), verify that the most frequent queries (like selecting recent history or listing orders) are efficient. Also test concurrent usage: how does the system behave with multiple simultaneous requests? Use a tool or scripts to simulate load and ensure no obvious bottlenecks or memory leaks.
- **Deployment Strategy:** Finalize the deployment approach and documentation. Although initially deferred ⁷⁷ ⁷⁸, now is the time to update `docs/DEPLOYMENT_STRATEGY.md` with instructions for deploying v2.2 to a production environment. This should cover setting up the

database (running migrations in prod), configuring environment variables for production (ensuring all keys are provided, Safe Mode maybe turned off if not desired in prod), setting up OAuth Redirect URI for the production domain, and how to use Docker or other infrastructure to host the app. Possibly create a docker-compose file for production that includes the app and a Prometheus instance, etc. If feasible, prepare a staging environment to do a trial deployment.

- **Documentation & Final Review:** Complete any remaining documentation: update the README with any new env vars or features, update the RUNBOOK if new troubleshooting or steps emerged, and create the **MASTER_STRATEGY v2.2** (this document) in the repo for reference. Also, update the risk register – by this point, the only in-progress items (DB consolidation) should be resolved, and we can mark all major risks as addressed. Conduct a final code review (especially security-critical code like auth and data import) to catch any issues.

Once Block 4 tasks are done, we will have a **production candidate v2.2** ready to release. At that stage, all P1/P2 risks will be mitigated, and the system will be fully functional with persistent data, user accounts, and multi-agent capabilities.

Future Ideas (Post-v2.2)

Looking beyond the immediate scope, a few enhancements could be explored after v2.2 if time allows or in future versions: multi-language support (allow the Chef's Mind AI interface to be in English, etc.), a mobile-friendly UI or simple mobile app, adding more agents (for example, a *Marketing Agent* for social media content), and improving the intelligence of each agent with fine-tuned models or additional domain knowledge. These are not part of the current plan but are noted for potential follow-up.

With the completion of Blocks 3 and 4, Chef's Mind AI v2.2 will be ready for demonstration and initial production use. All core features (multi-agent chat, integrated analytics, content generation, data management) will be working together. Team members should follow the guidelines in the next section when interacting with the Kilo Code AI assistant to ensure smooth development as we finalize this project.

Kilo Code Interaction Guidelines (Summary)

Kilo Code is the AI pair-programmer and orchestrator assisting in our development (integrated via VS Code). To use it effectively and safely, the team should follow these communication rules:

- **Use Clear, Stepwise Instructions:** When assigning a task to Kilo, break it into concrete steps or a specific outcome. The Kilo Orchestrator will decompose complex tasks into a structured plan (often following Plan → Do → Review → Log phases for major changes). Always focus on *what* needs to be done rather than *how*, and let Kilo propose the solution.
- **Safe Changes & Confirmation:** Kilo operates in a sandboxed manner – it will not write to files that are restricted (see `.kilocode/rules/restricted_files.md`) and will not execute destructive actions without confirmation. For any database migration, file deletion, or critical operation, Kilo will request a confirmation (aligning with Safe Mode policies). Be prepared to provide the `X-Confirm-Code` or a manual “go-ahead” in chat when Kilo prompts for it, after reviewing the proposed diff or action.
- **Append-Only Documentation:** When asking Kilo to update documentation or logs, remember that it adheres to an append-only rule. Kilo will add new **Addendum** sections or log entries rather than editing historical text ⁷⁹ ⁸⁰. Do not instruct Kilo to rewrite or remove past entries – instead, have it note changes in a new section. This ensures we maintain a traceable history.

- **Leverage Roles and Memory:** Kilo has multiple internal roles (Architect, Coder, Debugger, Documentor). You can prompt it to take a specific perspective, e.g., “As **Coder**, implement the new endpoint X” or “As **Debugger**, run and fix failing tests.” It also has access to a **Memory Bank** of project context (architecture, product goals, technical constraints). Use this to your advantage: reference key context files if needed (for example, remind it of `architecture_review.md` or similar) so it stays aligned with project vision.
- **Review Diffs and Outputs:** After Kilo provides code changes, always review the `diff` or the described modifications. Kilo will log changes to `ACTION_LOG.md` with a summary of affected files ⁸¹. Verify these changes solve the intended problem and do not introduce issues elsewhere. Only after careful review and testing should changes be committed. Kilo’s suggestions are helpful, but final responsibility lies with us – treat it like a junior developer’s work that needs code review.
- **Stay In Scope & Use Iteration:** Keep each interaction focused. If a task is large, Kilo will likely break it down – allow it to complete the plan step by step. Avoid jumping to unrelated requests mid-process to prevent confusion. If the output isn’t as expected, calmly clarify or correct the instruction and let Kilo try again. Iteration is normal; provide feedback like “That’s not quite right, please adjust the function to do Y.”
- **Maintain Security:** Do not ask Kilo to print or expose sensitive credentials or to bypass security features. It is configured to refuse actions against the rules (for instance, it won’t open `.env` files or other secret storage). Also, avoid requesting Kilo to access external internet resources or any file outside the repository scope – it’s confined to the project. If you need external info (like a library doc), fetch it yourself and feed the relevant snippet to Kilo instead.

By following these guidelines, we ensure that Kilo Code remains a powerful ally in development without compromising project integrity or security. Keep communication professional and precise, and the collaboration with the AI will continue to accelerate our development workflow.

Sources: Key project artifacts and documents were referenced in compiling this strategy ⁸² ⁸³. For detailed implementation specifics, see the repository files (e.g., `server/`, `docs/`) and prior checkpoint reports. This MASTER_STRATEGY v2.2 is to be kept as a living document – update the JSON manifest version below in parallel with any significant changes.

MASTER_STRATEGY_v2.2.json (Structured Manifest)

```
{
  "project": "Chef's Mind AI",
  "version": "2.2",
  "architecture": {
    "layers": [
      "Orchestrator API",
      "Agents (Chef, Accountant, Researcher, Media Studio)",
      "QA-Gate (middleware)",
      "External Providers (LLM, Google API, etc.)"
    ],
    "agents": [
      { "name": "Chef", "domain": "Culinary & Recipe", "status": "active" },
      { "name": "Accountant", "domain": "Finance & Procurement", "status": "active" },
      { "name": "Researcher", "domain": "Analytics & Research", "status":
```

```

"active" },
  { "name": "Media Studio", "domain": "Image/Video Generation",
"status": "active" },
  { "name": "QA-Gate", "domain": "Answer Validation", "status":
"integrated" }
]
},
"authorization": {
  "roles": [ "admin", "chef", "accountant" ],
  "rbac_policy": {
    "admin": ["admin", "chef", "accountant"],
    "chef": ["chef"],
    "accountant": ["accountant"]
  },
  "protected_endpoints": [
    "/api/import/*",
    "/api/db/*",
    "/api/media/*",
    "/api/calendar/*"
  ],
  "oauth_provider": "Google",
  "oauth_scopes": [ "email", "profile", "openid", "https://
www.googleapis.com/auth/calendar", "https://www.googleapis.com/auth/
documents", "https://www.googleapis.com/auth/spreadsheets" ],
  "safe_mode": {
    "enabled": true,
    "confirm_header": "X-Confirm-Code",
    "env_variable": "CONFIRM_CODE"
  }
},
"environment": {
  "dev_ports": {
    "api_backend": 5003,
    "frontend_primary": 3000,
    "frontend_backup": 3001,
    "database": 5432,
    "prometheus": 9090
  },
  "startup_order": [ "database", "backend_api", "frontend" ],
  "env_files": [ ".env.example", ".env.local", "frontend-
enhanced/.env.local" ],
  "tech_stack": {
    "backend": "Node.js 20 + Express + TypeScript",
    "frontend": "Next.js 14 (React 18)",
    "database": "PostgreSQL 15 + Drizzle ORM",
    "ai_providers": "OpenAI API, Google Cloud (Vertex AI), DALL·E 3"
  }
},
"directories": {
  "server": "Express backend source (routes, middleware, services, graph
logic)",

```

```

    "frontend-enhanced": "Next.js frontend application",
    "shared": "Shared types and DB schema definitions",
    "drizzle": "Database migration files",
    "tests": "Unit, integration, E2E tests",
    "docs": "Documentation (architecture, checkpoints, design docs)",
    "reports": "Logs, action log, and artifacts of changes"
  },
  "current_status": {
    "completion_pct": 92,
    "last_checkpoint": "v2.1.4",
    "stable": true,
    "notable_features": [
      "Intent-based agent routing with 95% accuracy",
      "All critical routes secured by RBAC and Safe Mode",
      "Prometheus metrics & alerts configured",
      "OAuth login integrated (Google OAuth2)",
      "Backup & restore functionality available"
    ],
    "known_issues": [
      "Windows/WSL file lock quirks in dev (documented workarounds)",
      "Heuristic agent routing (planned LLM improvement)",
      "Minor UI polish needed for new features (history, admin panel)"
    ]
  },
  "api_endpoints": {
    "public": [
      "GET /health",
      "POST /api/enhanced-agent/chat",
      "POST /api/media/image/generate",
      "POST /api/media/video/generate"
    ],
    "protected": [
      "POST /api/import/upload (admin + confirm)",
      "POST /api/db/backup (admin + confirm)",
      "GET /api/db/backups (admin)",
      "POST /api/db/restore (admin + confirm)",
      "POST /api/calendar/payment (admin + confirm)",
      "POST /api/calendar/delivery (admin + confirm)",
      "POST /api/calendar/followup (admin + confirm)"
    ],
    "auth": [
      "GET /auth/google/start (public, begins OAuth flow)",
      "GET /auth/google/callback (OAuth redirect URI)",
      "GET /auth/google/status (returns auth status JSON)",
      "POST /auth/google/logout (clears session)"
    ]
  },
  "roadmap": {
    "block3_data_persistence": {
      "status": "in_progress",
      "tasks": [

```

```

    "Chat history database & API (/api/chat/history)",
    "Orders & Suppliers CRUD endpoints + UI",
    "Generalized calendar create endpoint",
    "Backup/Restore UI integration",
    "Docs: Data persistence design doc",
    "New checkpoint v2.1.5 after completion"
  ],
  "eta": "2025-11-22"
},
"block4_finalization": {
  "status": "pending",
  "tasks": [
    "Migrate any remaining DDL to Drizzle migrations",
    "Finalize OAuth flow (testing & error handling)",
    "Integration tests for new features (history, orders, calendar)",
    "Performance tuning (DB indexes, load test)",
    "Deployment documentation & environment setup",
    "Final code review and risk assessment"
  ],
  "eta": "2025-11-30"
}
}
}
}

```

1 2 chefs_mind_ai_master_checkpoint_v_2025_10_12.md

https://github.com/veysitutar88/Chefs-Mind-AI/blob/72c41aa3b73775e05a99ff054412d4ed158c77c9/chefs_mind_ai_master_checkpoint_v_2025_10_12.md

3 4 11 30 33 46 61 71 72 73 74 76 83 risk_register.md

https://github.com/veysitutar88/Chefs-Mind-AI/blob/72c41aa3b73775e05a99ff054412d4ed158c77c9/docs/risk_register.md

5 6 7 10 53 75 82 progress_report_v_2_1.md

<file://file-NokAYfpe7av828ah66SdzZ>

8 9 quality_control.ts

https://github.com/veysitutar88/Chefs-Mind-AI/blob/72c41aa3b73775e05a99ff054412d4ed158c77c9/server/graph/nodes/quality_control.ts

12 13 14 15 16 rbac.ts

<https://github.com/veysitutar88/Chefs-Mind-AI/blob/72c41aa3b73775e05a99ff054412d4ed158c77c9/server/middleware/rbac.ts>

17 18 21 22 23 67 68 69 70 auth.google.ts

<https://github.com/veysitutar88/Chefs-Mind-AI/blob/72c41aa3b73775e05a99ff054412d4ed158c77c9/server/routes/auth.google.ts>

19 20 G1_OAUTH_FIX_SUMMARY_2025-11-04.md

https://github.com/veysitutar88/Chefs-Mind-AI/blob/72c41aa3b73775e05a99ff054412d4ed158c77c9/reports/G1_OAUTH_FIX_SUMMARY_2025-11-04.md

24 25 safeMode.ts

<https://github.com/veysitutar88/Chefs-Mind-AI/blob/72c41aa3b73775e05a99ff054412d4ed158c77c9/server/middleware/safeMode.ts>

26 27 28 29 59 60 **dbadmin.ts**

<https://github.com/veysitutar88/Chefs-Mind-AI/blob/72c41aa3b73775e05a99ff054412d4ed158c77c9/server/routes/dbadmin.ts>

31 34 35 36 41 42 43 44 45 51 52 **RUNBOOK.md**

<https://github.com/veysitutar88/Chefs-Mind-AI/blob/72c41aa3b73775e05a99ff054412d4ed158c77c9/RUNBOOK.md>

32 54 **context.md**

<https://github.com/veysitutar88/Chefs-Mind-AI/blob/72c41aa3b73775e05a99ff054412d4ed158c77c9/.kilocode/rules/memory-bank/context.md>

37 38 39 40 47 48 49 50 79 80 81 **README.md**

<https://github.com/veysitutar88/Chefs-Mind-AI/blob/72c41aa3b73775e05a99ff054412d4ed158c77c9/README.md>

55 56 57 58 **importer.ts**

<https://github.com/veysitutar88/Chefs-Mind-AI/blob/72c41aa3b73775e05a99ff054412d4ed158c77c9/server/routes/importer.ts>

62 63 64 **calendar.ts**

<https://github.com/veysitutar88/Chefs-Mind-AI/blob/72c41aa3b73775e05a99ff054412d4ed158c77c9/server/routes/calendar.ts>

65 66 **google-mcp.ts**

<https://github.com/veysitutar88/Chefs-Mind-AI/blob/72c41aa3b73775e05a99ff054412d4ed158c77c9/server/services/google-mcp.ts>

77 78 **PR_Block0_InfraFix.md**

<file:///file-6JFGPpZzZVKBYqQn8jFp49>